3D Room Customizer

Complete code explanation and source reference

> **Updated feature coverage:** built-in metal arm chair and user .glb furniture importing.
>
> This guide is generated from the current workspace source. It explains the architecture, data flow, functions, validation, caching, placement, and complete source appendix.

Updated: 5 November 2026. Binary GLB files are described but not printed into the appendix.

Contents

1. Architecture and new features
2. Page behavior
3. Function index
4. Complete source appendix

Feature flow

```
Built-in CATALOG ──────┐
                       ├─ buildFurniturePivot — placement — scene/store
Import button → File ──┘
  validate .glb/non-empty
  arrayBuffer → GLTFLoader.parseAsync
  WeakMap cache → dynamic Add option
```

1. Architecture and data flow

Browser flow

- mainpage.html links to canvas.html or preset.html. Both workflows serialize their room choice to sessionStorage and navigate to index.html.
- index.html loads src/main.js. The import map resolves three and Three.js addons from node_modules.
- main.js composes room, rendering, furniture, physics, interaction, and UI modules; it remains the application entry point.

Room contract

- All room builders return a Group carrying userData.outline, pillars, wallThickness, and walls.
- The outline and pillars use floor coordinates {x, z}; collision and placement code consume this common contract.

Furniture workflow

- Built-in catalog entries load through buildFurniture(). Imported File objects load through buildFurnitureFromFile(). Both produce the same centered pivot Group.
- The Add menu creates built-in options from CATALOG and exposes a separate Import button. Selecting a file adds a dynamic option after successful parsing.
- Imported files must have a .glb extension and non-zero size. Invalid files receive a toast and never enter the menu.
- Imported GLB bytes are read with file.arrayBuffer(), parsed with GLTFLoader.parseAsync(data, ''), and cached in a WeakMap keyed by the File object.
- Both built-in and imported items go through the same placement search, scene insertion, store registration, dragging, rotation, collision, and removal behavior.

Sizing and validation

- buildFurniturePivot() clones the source, enables mesh shadows, applies optional rotation, measures a Box3, scales to entry.width or entry.height, and moves the model so its pivot is centered on the floor.
- Imported models with empty or non-finite geometry are rejected with 'The GLB file has no usable model geometry'.
- A failed imported parse removes its WeakMap cache entry so a later retry can try again.

Collision and rendering

- Furniture collision uses axis-aligned floor rectangles; fitsInRoom checks walls, outline containment, and pillars.
- findFreeSpot searches a configurable grid and prefers the valid candidate closest to the room center.
- The animation loop updates wall visibility, OrbitControls damping, and the renderer; resize updates camera projection and renderer size.

2. Page-by-page behavior

mainpage.html

Provides the two starting workflows and responsive landing-page presentation.

preset.html

updateDimensions() controls shape-specific inputs. Submit serializes FormData to presetRoom and redirects to index.html.

canvas.html and src/canvas.js

Coordinates snap to a 25-pixel grid. The first closed polygon is the room; later closed polygons are pillars. Measurements and previews are redrawn on pointer activity, then customRoom is saved.

index.html and src/main.js

The editor composes the current room, renderer, catalog menu, Import callback, collision placement, drag controls, and context menu. addFurniture() supports either catalog paths or File objects.

3. Function index

File	Line	Function
`preset.html`	61	`updateDimensions()`
`src/canvas.js`	35	`resizeCanvas()`
`src/canvas.js`	41	`drawMeasurement()`
`src/canvas.js`	70	`drawShape()`

File	Line	Function
src/canvas.js	95	redraw()
src/canvas.js	107	finishRoom()
src/core/scene.js	10	createScene()
src/custom_model.js	8	buildCustomRoom()
src/furniture/loader.js	9	loadModel()
src/furniture/loader.js	23	buildFurniture()
src/furniture/loader.js	29	buildFurnitureFromFile()
src/furniture/loader.js	53	buildFurniturePivot()
src/furniture/placement.js	6	findFreeSpot()
src/furniture/store.js	2	createFurnitureStore()
src/interaction/contextMenu.js	2	installContextMenu()
src/interaction/contextMenu.js	5	close()
src/interaction/contextMenu.js	9	open()
src/interaction/drag.js	6	installDrag()
src/interaction/drag.js	15	setPointer()
src/interaction/drag.js	21	pick()
src/interaction/drag.js	28	startDrag()
src/interaction/drag.js	38	stopDrag()
src/main.js	40	placeFurniture()
src/main.js	52	addFurniture()
src/main.js	64	importFurniture()
src/main.js	95	animate()
src/physics/collision.js	11	footprint()
src/physics/collision.js	33	fitsInRoom()
src/physics/collision.js	54	isFree()
src/physics/collision.js	63	nudgeToFree()
src/physics/collision.js	88	tryOffset()
src/physics/collision.js	102	moveWithCollision()
src/physics/geometry.js	2	pointInPolygon()

File	Line	Function
src/physics/geometry.js	18	segmentHitsRect()
src/physics/geometry.js	44	polygonHitsRect()
src/physics/geometry.js	52	rectsOverlap()
src/room/defaultRoom.js	4	buildDefaultRoom()
src/room/defaultRoom.js	6	addPart()
src/room/defaultRoom.js	7	addWall()
src/room/loadRoom.js	6	loadRoom()
src/room/presetRoom.js	5	rectanglePoints()
src/room/presetRoom.js	16	cylinderPoints()
src/room/presetRoom.js	32	buildPresetRoom()
src/room/wallVisibility.js	8	updateWallVisibility()
src/ui/addMenu.js	2	buildAddMenu()
src/ui/addMenu.js	29	addEntry()
src/ui/toast.js	4	showMessage()

4. Complete source appendix

mainpage.html

> Landing page for choosing custom drawing or preset dimensions.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Roomly | Design your room</title>
    <style>
        :root {
            color-scheme: dark;
            --ink: #f7f4ef;
            --muted: #b7b2aa;
            --accent: #ffb86b;
            --accent-dark: #d97638;
            --panel: rgba(31, 29, 34, 0.78);
        }

        * { box-sizing: border-box; }
        body {
            margin: 0;
            min-height: 100vh;
            display: grid;
            place-items: center;
            color: var(--ink);
            font-family: Inter, ui-sans-serif, system-ui, -apple-system, sans-serif;
            background:
                radial-gradient(circle at 75% 15%, rgba(255, 184, 107, .25), transparent 30%),
                linear-gradient(135deg, #17151a 0%, #27232a 55%, #3b2b2c 100%);
        }

        main {
            width: min(1120px, 92vw);
            padding: clamp(2rem, 6vw, 5rem) 0;
        }
        .brand { color: var(--accent); font-weight: 800; letter-spacing: .16em; text-transform: uppercase; }
        h1 { max-width: 720px; margin: 1.25rem 0 1rem; font-size: clamp(3rem, 8vw, 6.7rem); line-height: .94;
letter-spacing: -.07em; }
        .intro { max-width: 550px; margin: 0 0 3rem; color: var(--muted); font-size: 1.1rem; line-height: 1.6; }
        .choices { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 1rem; max-width: 850px; }
        .choice {
            min-height: 225px; padding: 1.7rem; display: flex; flex-direction: column; justify-content: space-
between;
            color: inherit; text-decoration: none; border: 1px solid rgba(255,255,255,.13); border-radius: 24px;
            background: var(--panel); backdrop-filter: blur(14px); transition: transform .2s, border-color .2s,
background .2s;
        }
        .choice:hover, .choice:focus-visible { transform: translateY(-6px); border-color: var(--accent);
background: rgba(55, 48, 51, .9); outline: none; }
        .choice-number { color: var(--accent); font-size: .82rem; font-weight: 700; letter-spacing: .1em; }
        h2 { margin: 1.8rem 0 .45rem; font-size: 1.6rem; }
        .choice p { margin: 0; color: var(--muted); line-height: 1.5; }
        .arrow { align-self: flex-end; color: var(--accent); font-size: 1.8rem; }
        @media (max-width: 650px) { .choices { grid-template-columns: 1fr; } .choice { min-height: 190px; } }
    </style>
</head>
<body>
    <main>
        <div class="brand">Roomly</div>
        <h1>Make space for your ideas.</h1>
        <p class="intro">Design a room that feels like yours. Start with a quick preset or sketch the space
exactly as you imagine it.</p>
        <section class="choices" aria-label="Choose how to start">
            <a class="choice" href="canvas.html">
                <span class="choice-number">01 / CUSTOM</span>
                <div><h2>Draw your room</h2><p>Sketch the outline and tell us about any pillars inside it.</p>
</div>
                <span class="arrow" aria-hidden="true">↗</span>
            </a>
            <a class="choice" href="preset.html">
```

```
  63:                 <span class="choice-number">02 / QUICK START</span>
  64:                 <div><h2>Choose a preset</h2><p>Pick a room shape and dimensions, then start arranging furniture.
</p></div>
  65:                 <span class="arrow" aria-hidden="true">↗</span>
  66:             </a>
  67:         </section>
  68:     </main>
  69: </body>
  70: </html>
```

preset.html

> Preset form; dynamically enables the relevant dimensions and saves presetRoom to sessionStorage.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Roomly | Choose a preset</title>
    <style>
        :root { --ink:#f7f4ef; --muted:#a9a39d; --accent:#ffb86b; --panel:#252229; }
        * { box-sizing: border-box; }
        body { margin:0; min-height:100vh; display:grid; place-items:center; color:var(--ink); font-family:Inter,system-ui,sans-serif; background:linear-gradient(140deg,#17151a,#3a2a2d); }
        .card { width:min(680px,92vw); padding:clamp(1.5rem,5vw,3.5rem); border:1px solid rgba(255,255,255,.13); border-radius:28px; background:rgba(31,29,34,.86); box-shadow:0 24px 80px rgba(0,0,0,.25); }
        .back { color:var(--muted); text-decoration:none; } .back:hover { color:var(--accent); }
        h1 { margin:2.5rem 0 .6rem; font-size:clamp(2.2rem,6vw,4rem); letter-spacing:-.06em; }
        .subtitle { margin:0 0 2rem; color:var(--muted); line-height:1.5; }
        form { display:grid; gap:1.3rem; }
        .field-row { display:grid; grid-template-columns:1fr 1fr; gap:1rem; }
        label { display:grid; gap:.55rem; color:var(--muted); font-size:.9rem; font-weight:600; }
        select, input { width:100%; padding:1rem; color:var(--ink); border:1px solid #514951; border-radius:12px; background:var(--panel); font:inherit; }
        [hidden] { display:none !important; }
        select:focus, input:focus { border-color:var(--accent); outline:2px solid rgba(255,184,107,.2); }
        button { margin-top:.8rem; padding:1rem 1.2rem; color:#271d19; border:0; border-radius:12px; background:var(--accent); font:700 1rem inherit; cursor:pointer; }
        button:hover { background:#ffc98f; } @media(max-width:520px){.field-row{grid-template-columns:1fr;}}
    </style>
</head>
<body>
    <main class="card">
        <a class="back" href="mainpage.html">← Back</a>
        <h1>Start with a shape.</h1>
        <p class="subtitle">Choose a foundation for your room. You can refine the space as you arrange it.</p>
        <form id="preset-form" action="index.html">
            <label>Room shape
                <select name="shape" id="shape">
                    <option value="rectangle">Rectangle</option>
                    <option value="square">Square</option>
                    <option value="cylinder">Cylinder</option>
                </select>
            </label>
            <div class="field-row" id="rectangular-dimensions">
                <label>Width <input type="number" name="width" value="10" min="1" step="0.5"> </label>
                <label id="depth-field">Length <input type="number" name="depth" value="10" min="1" step="0.5">
</label>
            </div>
            <div id="square-dimensions" hidden>
                <label>Length <input type="number" name="length" value="10" min="1" step="0.5" disabled> </label>
            </div>
            <div id="cylinder-dimensions" hidden>
                <label>Radius <input type="number" name="radius" value="5" min="1" step="0.5" disabled> </label>
            </div>
            <button type="submit">Open room editor <span aria-hidden="true">→</span></button>
        </form>
    </main>
    <script>
        const presetForm = document.getElementById("preset-form");
        const shapeSelect = document.getElementById("shape");
        const rectangularDimensions = document.getElementById("rectangular-dimensions");
        const squareDimensions = document.getElementById("square-dimensions");
        const cylinderDimensions = document.getElementById("cylinder-dimensions");
        const rectangularInputs = rectangularDimensions.querySelectorAll("input");
        const squareInput = squareDimensions.querySelector("input");
        const cylinderInput = cylinderDimensions.querySelector("input");

        function updateDimensions() {
            const shape = shapeSelect.value;
            const isSquare = shape === "square";
```

```
64:             const isCylinder = shape === "cylinder";
65:             const showRectangular = !isSquare && !isCylinder;
66:
67:             rectangularDimensions.hidden = !showRectangular;
68:             squareDimensions.hidden = !isSquare;
69:             cylinderDimensions.hidden = !isCylinder;
70:             rectangularInputs.forEach((input) => {
71:                 input.disabled = !showRectangular;
72:                 input.required = showRectangular;
73:             });
74:             squareInput.disabled = !isSquare;
75:             squareInput.required = isSquare;
76:             cylinderInput.disabled = !isCylinder;
77:             cylinderInput.required = isCylinder;
78:         }
79:
80:         shapeSelect.addEventListener("change", updateDimensions);
81:         updateDimensions();
82:
83:         presetForm.addEventListener("submit", (event) => {
84:             event.preventDefault();
85:             const formData = new FormData(presetForm);
86:             sessionStorage.setItem("presetRoom", JSON.stringify(Object.fromEntries(formData)));
87:             window.location.href = presetForm.action;
88:         });
89:     </script>
90: </body>
91: </html>
```

canvas.html

> Canvas-based custom room drawing page with optional pillar setup.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Roomly | Draw your room</title>
    <style>
        html, body { margin: 0; height: 100%; overflow: hidden; }
        body { background: #f4eee5; font-family: Inter, system-ui, sans-serif; }
        canvas {
            display: block;
            background-color: #fffdf9;
            background-image:
                linear-gradient(rgba(184, 91, 48, .13) 1px, transparent 1px),
                linear-gradient(90deg, rgba(184, 91, 48, .13) 1px, transparent 1px);
            background-size: 50px 50px;
            cursor: crosshair;
        }
        .instructions { position: fixed; top: 24px; left: 24px; max-width: 320px; padding: 18px 20px; color:
#332b27; border: 1px solid rgba(87,65,48,.15); border-radius: 16px; background: rgba(255,253,249,.9); box-shadow: 0 10px
30px rgba(64,43,24,.12); pointer-events: none; }
        .instructions strong { display: block; margin-bottom: 6px; color: #b85b30; }
        .instructions span { color: #716961; font-size: .9rem; line-height: 1.4; }
        .back { position: fixed; right: 24px; top: 24px; padding: 10px 14px; color: #55483f; border-radius: 10px;
background: rgba(255,253,249,.9); text-decoration: none; }
        .setup { position: fixed; inset: 0; display: grid; place-items: center; background: rgba(244,238,229,.92);
}
        .setup-card { width: min(420px, 88vw); padding: 28px; color: #332b27; border-radius: 22px; background:
#fffdf9; box-shadow: 0 20px 60px rgba(64,43,24,.18); }
        .setup-card h1 { margin: 0 0 8px; font-size: 1.7rem; }
        .setup-card p { margin: 0 0 22px; color: #716961; line-height: 1.45; }
        .setup-card label { display: grid; gap: 8px; color: #55483f; font-weight: 600; }
        .setup-card select { padding: 11px; border: 1px solid #d5c9bd; border-radius: 10px; background: #fff;
font: inherit; }
        .setup-card button { width: 100%; margin-top: 20px; padding: 12px; color: #fff; border: 0; border-radius:
10px; background: #b85b30; font: 700 1rem inherit; cursor: pointer; }
        .setup-card button:hover { background: #984724; }
        #pillar-count-wrap[hidden] { display: none; }
    </style>
</head>
<body>
    <section class="setup" id="pillar-setup">
        <form class="setup-card" id="pillar-form">
            <h1>One quick detail.</h1>
            <p>Does your room have pillars inside it? We'll use your answer to guide the drawing.</p>
            <label for="has-pillars">Pillars inside the room?
                <select id="has-pillars" name="hasPillars">
                    <option value="no">No</option>
                    <option value="yes">Yes</option>
                </select>
            </label>
            <label id="pillar-count-wrap" for="pillar-count" hidden>How many pillars?
                <select id="pillar-count" name="pillarCount">
                    <option value="1">1 pillar</option>
                    <option value="2">2 pillars</option>
                    <option value="3">3 pillars</option>
                    <option value="4">4 pillars</option>
                    <option value="5">5 pillars</option>
                </select>
            </label>
            <button type="submit">Start drawing <span aria-hidden="true">→</span></button>
        </form>
    </section>
    <div class="instructions"><strong>Draw your room</strong><span>Click points to create the outline.
Measurements are shown in metres as you draw. Click the first point again to close it, then draw each pillar.</span>
</div>
    <a class="back" href="mainpage.html">Exit</a>
    <canvas id="draw"></canvas>
```

```
    <script type="importmap">
    {
        "imports": {
            "three": "./node_modules/three/build/three.module.js",
            "three/addons/": "./node_modules/three/examples/jsm/"
        }
    }
    </script>
    <script type="module" src="./src/canvas.js"></script>
</body>
</html>
```

index.html

> Three.js editor shell and ES-module import map.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>three js application</title>

    <script type="importmap">
{
  "imports": {
    "three": "./node_modules/three/build/three.module.js",
    "three/addons/": "./node_modules/three/examples/jsm/"
  }
}
</script>
<script type="module" src="./src/main.js"></script>
    <style>
        #scene-container { width: 100%; height: 100%; }
        body{
            height:100%;
            margin: 0;
            overflow:hidden;

            text-align: center;
            font-size: 12px;
            font-family: sans-serif;

            color: #444;
        }

        h1{
            position: absolute;
            top: 14px;
            left: 0;
            width: 100%;
            margin: 0;

            font-size: 22px;
            font-weight: 600;
            color: #1d3557;

            z-index: 5;            /* above the canvas */
            pointer-events: none;  /* clicks pass through to the 3D scene */
        }
        #scene-container{
            position : absolute;
            width: 100%;
            height:100%;
            background-color: skyblue;
        }


    </style>
</head>
<body>
    <h1>3D Room Customizer, see it before you buy it!</h1>    <div id="scene-container">
        <!-- ou canvas will be inserted here !-->
    </div>
</body>
</html>
```

src/main.js

> Composition root for room loading, rendering, furniture, import, placement, and interaction.

```
import { loadRoom } from "./room/loadRoom.js";
import { createScene } from "./core/scene.js";
import { CATALOG } from "./furniture/catalog.js";
import { buildFurniture, buildFurnitureFromFile } from "./furniture/loader.js";
import { findFreeSpot } from "./furniture/placement.js";
import { createFurnitureStore } from "./furniture/store.js";
import { updateWallVisibility } from "./room/wallVisibility.js";
import { installDrag } from "./interaction/drag.js";
import { installContextMenu } from "./interaction/contextMenu.js";
import { buildAddMenu } from "./ui/addMenu.js";
import { showMessage } from "./ui/toast.js";

const container = document.getElementById("scene-container");
const room = loadRoom();
const {
  renderer,
  scene,
  camera,
  orbit,
  bounds,
  centerX,
  centerZ,
} = createScene(container, room);
console.log("room data:", {
  outline: room.userData.outline,
  roomPillars: room.userData.pillars ?? [],
  roomWallT: room.userData.wallThickness ?? 0.12,
});
const collisionRoom = {
  outline: room.userData.outline,
  pillars: room.userData.pillars ?? [],
  wallThickness: room.userData.wallThickness ?? 0.12,
  bounds,
  centerX,
  centerZ,
};
const store = createFurnitureStore();
const furniture = store.items;

function placeFurniture(pivot, entry) {
  const spot = findFreeSpot(pivot, collisionRoom, furniture);
  if (!spot) {
    showMessage(`Not enough space for ${entry.label}`);
    return;
  }
  pivot.rotation.y = spot.rot;
  pivot.position.set(spot.x, 0, spot.z);
  scene.add(pivot);
  store.add(pivot);
}

async function addFurniture(entry) {
  try {
    const pivot = entry.file
      ? await buildFurnitureFromFile(entry.file, entry)
      : await buildFurniture(entry);
    placeFurniture(pivot, entry);
  } catch (error) {
    console.error(`Failed to load ${entry.label}:`, error);
    showMessage(`Could not load ${entry.label}`);
  }
}

async function importFurniture(file, addEntry) {
  if (!file.name.toLowerCase().endsWith(".glb")) {
    showMessage("Choose a .glb file");
    return;
  }
```

```
  69:   if (file.size === 0) {
  70:     showMessage("The selected file is empty");
  71:     return;
  72:   }
  73:
  74:   const label = file.name.replace(/\.glb$/i, "") || file.name;
  75:   const entry = {
  76:     id: `imported-${Date.now()}-${file.name}`,
  77:     label,
  78:     file,
  79:     width: 2,
  80:   };
  81:   try {
  82:     const pivot = await buildFurnitureFromFile(file, entry);
  83:     addEntry(entry);
  84:     placeFurniture(pivot, entry);
  85:   } catch (error) {
  86:     console.error(`Failed to load ${file.name}:`, error);
  87:     showMessage(`Could not load ${file.name}`);
  88:   }
  89: }
  90:
  91: buildAddMenu(CATALOG, addFurniture, importFurniture);
  92: const drag = installDrag({ renderer, camera, orbit, scene, furniture, room: collisionRoom });
  93: installContextMenu({ renderer, furniture, scene, drag, store });
  94:
  95: function animate() {
  96:   requestAnimationFrame(animate);
  97:   updateWallVisibility(room, camera);
  98:   orbit.update();
  99:   renderer.render(scene, camera);
 100: }
 101: animate();
 102: window.addEventListener("resize", () => {
 103:   camera.aspect = container.clientWidth / container.clientHeight;
 104:   camera.updateProjectionMatrix();
 105:   renderer.setSize(container.clientWidth, container.clientHeight);
 106: });
```

src/config.js

> Shared grid, collision tolerance, and helper configuration.

```
1: /** Grid resolution, in room units, used to search for furniture placement. */
2: export const GRID_STEP = 0.25;
3: /** Small positional tolerance, in room units, used for collision checks. */
4: export const EPS = 1e-4;
5: /** Whether scene axes, grid, and light helpers are shown. */
6: export const SHOW_HELPERS = true;
```

src/custom_model.js

> Converts point arrays into a Three.js room, walls, floor, pillars, and collision metadata.

```
 1: import * as THREE from 'three';
 2:
 3: const SCALE = 0.02;
 4: const WALL_H = 2.5;
 5: const WALL_T = 0.12;
 6:
 7: /** Builds a Three.js room group from canvas point arrays and pillar polygons in canvas coordinates. */
 8: export function buildCustomRoom(points, pillars) {
 9:     const group = new THREE.Group();
10:     const walls = [];
11:
12:     const xs = points.map((point) => point.x);
13:     const ys = points.map((point) => point.y);
14:     const cx = (Math.min(...xs) + Math.max(...xs)) / 2;
15:     const cy = (Math.min(...ys) + Math.max(...ys)) / 2;
16:     const toVector = (point) =>
17:         new THREE.Vector2((point.x - cx) * SCALE, (point.y - cy) * SCALE);
18:     const area = points.reduce((sum, point, index) => {
19:         const next = points[(index + 1) % points.length];
20:         return sum + point.x * next.y - next.x * point.y;
21:     }, 0);
22:     const outwardDirection = area >= 0 ? 1 : -1;
23:
24:     const floorShape = new THREE.Shape(points.map(toVector));
25:     pillars.forEach((pillar) => {
26:         floorShape.holes.push(new THREE.Path(pillar.map(toVector)));
27:     });
28:
29:     const floor = new THREE.Mesh(
30:         new THREE.ShapeGeometry(floorShape),
31:         new THREE.MeshStandardMaterial({ color: 0xbfae93, side: THREE.DoubleSide })
32:     );
33:     floor.rotation.x = Math.PI / 2;
34:     floor.receiveShadow = true;
35:     group.add(floor);
36:
37:     const wallMaterial = new THREE.MeshStandardMaterial({ color: 0xf0f0f0 });
38:     for (let i = 0; i < points.length; i++) {
39:         const a = toVector(points[i]);
40:         const b = toVector(points[(i + 1) % points.length]);
41:         const dx = b.x - a.x;
42:         const dz = b.y - a.y;
43:         const length = Math.hypot(dx, dz);
44:
45:         const wall = new THREE.Mesh(
46:             new THREE.BoxGeometry(length + WALL_T, WALL_H, WALL_T),
47:             wallMaterial
48:         );
49:         wall.position.set((a.x + b.x) / 2, WALL_H / 2, (a.y + b.y) / 2);
50:         wall.rotation.y = -Math.atan2(dz, dx);
51:         wall.receiveShadow = true;
52:         group.add(wall);
53:         walls.push({
54:             mesh: wall,
55:             normal: new THREE.Vector3(
56:                 outwardDirection * dz,
57:                 0,
58:                 outwardDirection * -dx
59:             ).normalize()
60:         });
61:     }
62:
63:     const pillarMaterial = new THREE.MeshStandardMaterial({ color: 0xe08a2e });
64:     pillars.forEach((pillar) => {
65:         const mesh = new THREE.Mesh(
66:             new THREE.ExtrudeGeometry(new THREE.Shape(pillar.map(toVector)), {
67:                 depth: WALL_H,
68:                 bevelEnabled: false
```

```
            }),
            pillarMaterial
        );
        mesh.rotation.x = Math.PI / 2;
        mesh.position.y = WALL_H;
        mesh.castShadow = true;
        mesh.receiveShadow = true;
        group.add(mesh);
    });

    group.userData.walls = walls;
    group.userData.pillars = pillars.map((pillar) =>
        pillar.map(toVector).map((v) => ({ x: v.x, z: v.y }))
    );
    group.userData.outline = points.map(toVector).map((v) => ({ x: v.x, z: v.y }));
    group.userData.wallThickness = WALL_T;
    return group;
}
```

src/canvas.js

> Custom-room drawing state machine, snapping, measurement previews, and persistence.

```
const canvas = document.getElementById('draw');
const ctx = canvas.getContext('2d');

const GRID = 25;
const snap = v => Math.round(v / GRID) * GRID;
const pixelsToMetres = pixels => pixels / GRID;

let numPillars = 0;

let points = [];          // room outline
let isClosed = false;     // is the outline finished?

let pillars = [];         // FINISHED pillars: a list of lists, e.g. [[p,p,p,p], [p,p,p,p]]
let currentPillar = [];   // the pillar being drawn right now
let mousePoint = null;

const setup = document.getElementById('pillar-setup');
const pillarForm = document.getElementById('pillar-form');
const hasPillars = document.getElementById('has-pillars');
const pillarCountWrap = document.getElementById('pillar-count-wrap');
const pillarCount = document.getElementById('pillar-count');

hasPillars.addEventListener('change', () => {
    pillarCountWrap.hidden = hasPillars.value !== 'yes';
});

pillarForm.addEventListener('submit', event => {
    event.preventDefault();
    numPillars = hasPillars.value === 'yes' ? Number.parseInt(pillarCount.value, 10) : 0;
    sessionStorage.setItem('numPillars', String(numPillars));
    setup.remove();
    redraw();
});

function resizeCanvas() {
    canvas.width = window.innerWidth;
    canvas.height = window.innerHeight;
    redraw();
}

function drawMeasurement(start, end, color, preview = false) {
    const distance = pixelsToMetres(Math.hypot(end.x - start.x, end.y - start.y) * 0.5);
    const label = `${distance.toFixed(1)} m`;
    const x = (start.x + end.x) / 2;
    const y = (start.y + end.y) / 2;

    ctx.save();
    ctx.font = '600 14px Inter, system-ui, sans-serif';
    const padding = 5;
    const textWidth = ctx.measureText(label).width;
    ctx.fillStyle = 'rgba(255, 253, 249, .9)';
    ctx.fillRect(x - textWidth / 2 - padding, y - 18, textWidth + padding * 2, 22);
    ctx.fillStyle = color;
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillText(label, x, y - 7);
    if (preview) {
        ctx.setLineDash([6, 5]);
        ctx.strokeStyle = color;
        ctx.lineWidth = 1;
        ctx.beginPath();
        ctx.moveTo(start.x, start.y);
        ctx.lineTo(end.x, end.y);
        ctx.stroke();
    }
    ctx.restore();
}

```

```
// Draw ONE shape (lines + dots) without clearing the canvas
function drawShape(list, closed, color) {
    if (list.length > 1) {
        ctx.beginPath();
        list.forEach((p, i) => i ? ctx.lineTo(p.x, p.y) : ctx.moveTo(p.x, p.y));
        if (closed) ctx.closePath();
        ctx.strokeStyle = color;
        ctx.lineWidth = 3;
        ctx.stroke();

        for (let i = 1; i < list.length; i++) {
            drawMeasurement(list[i - 1], list[i], color);
        }
        if (closed) {
            drawMeasurement(list[list.length - 1], list[0], color);
        }
    }
    ctx.fillStyle = color;
    list.forEach(p => {
        ctx.beginPath();
        ctx.arc(p.x, p.y, 5, 0, Math.PI * 2);
        ctx.fill();
    });
}

// Clear once, then draw everything
function redraw() {
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    drawShape(points, isClosed, 'black');
    pillars.forEach(pillar => drawShape(pillar, true, 'orange'));   // finished pillars
    drawShape(currentPillar, false, 'orange');                      // the one in progress

    const activeList = isClosed ? currentPillar : points;
    if (activeList.length > 0 && mousePoint) {
        drawMeasurement(activeList[activeList.length - 1], mousePoint, isClosed ? 'orange' : 'black', true);
    }
}

function finishRoom() {
    sessionStorage.removeItem('presetRoom');
    sessionStorage.setItem('customRoom', JSON.stringify({ points, pillars }));
    window.location.href = 'index.html';
}

canvas.addEventListener('click', e => {
    const p = { x: snap(e.clientX), y: snap(e.clientY) };

    // which list are we filling right now?
    const list = isClosed ? currentPillar : points;

    // clicked near the first point of THIS list => close it
    if (list.length > 2 && Math.hypot(p.x - list[0].x, p.y - list[0].y) < 12) {
        if (isClosed) {
            pillars.push(currentPillar);   // save the finished pillar
            currentPillar = [];            // start a fresh empty one
        } else {
            isClosed = true;
        }

        if (isClosed && pillars.length >= numPillars) {
            console.log('outline:', points);
            console.log('pillars:', pillars);
            finishRoom();
            return;
        }
        redraw();
        return;
    }

    list.push(p);
    redraw();
});

canvas.addEventListener('mousemove', e => {
    mousePoint = { x: snap(e.clientX), y: snap(e.clientY) };
    redraw();
});

```

```
canvas.addEventListener('mouseleave', () => {
    mousePoint = null;
    redraw();
});

window.addEventListener('resize', resizeCanvas);
resizeCanvas();
```

src/core/scene.js

> Renderer, camera, controls, lights, helpers, room bounds, and scene scaling.

```
 1: import {
 2:   AmbientLight, AxesHelper, CameraHelper, Color, DirectionalLight,
 3:   DirectionalLightHelper, GridHelper, PCFSoftShadowMap, PerspectiveCamera,
 4:   Scene, WebGLRenderer,
 5: } from "three";
 6: import { OrbitControls } from "three/addons/controls/OrbitControls.js";
 7: import { SHOW_HELPERS } from "../config.js";
 8:
 9: /** Creates the renderer, scene, camera, controls, and lighting for the given room bounds. */
10: export function createScene(container, room) {
11:   const renderer = new WebGLRenderer({ antialias: true });
12:   renderer.setSize(container.clientWidth, container.clientHeight);
13:   renderer.setPixelRatio(window.devicePixelRatio);
14:   renderer.shadowMap.enabled = true;
15:   renderer.shadowMap.type = PCFSoftShadowMap;
16:   container.append(renderer.domElement);
17:
18:   const scene = new Scene();
19:   scene.background = new Color("skyblue");
20:   const camera = new PerspectiveCamera(35, container.clientWidth / container.clientHeight, 0.1, 100);
21:   camera.position.set(14, 10, 18);
22:   const orbit = new OrbitControls(camera, renderer.domElement);
23:   orbit.target.set(0, 1.5, 0);
24:   orbit.enableDamping = true;
25:   orbit.update();
26:
27:   scene.add(new AmbientLight(0xffffff, 0.4));
28:   const directionalLight = new DirectionalLight(0xffffff, 1.5);
29:   directionalLight.position.set(4, 8, 3);
30:   directionalLight.castShadow = true;
31:   directionalLight.shadow.mapSize.set(2048, 2048);
32:   Object.assign(directionalLight.shadow.camera, {
33:     near: 0.5, far: 25, left: -8, right: 8, top: 8, bottom: -8,
34:   });
35:   scene.add(directionalLight);
36:
37:   if (SHOW_HELPERS) {
38:     scene.add(new AxesHelper(10));
39:     const gridHelper = new GridHelper(10, 10);
40:     gridHelper.position.y = 0.01;
41:     scene.add(gridHelper, new DirectionalLightHelper(directionalLight, 1));
42:     scene.add(new CameraHelper(directionalLight.shadow.camera));
43:   }
44:
45:   scene.add(room);
46:   const outline = room.userData.outline;
47:   const bounds = {
48:     minX: Math.min(...outline.map((p) => p.x)),
49:     maxX: Math.max(...outline.map((p) => p.x)),
50:     minZ: Math.min(...outline.map((p) => p.z)),
51:     maxZ: Math.max(...outline.map((p) => p.z)),
52:   };
53:   const centerX = (bounds.minX + bounds.maxX) / 2;
54:   const centerZ = (bounds.minZ + bounds.maxZ) / 2;
55:   const extent = Math.max(bounds.maxX - bounds.minX, bounds.maxZ - bounds.minZ);
56:   const scale = Math.max(1, extent / 10);
57:
58:   camera.position.set(14, 10, 18).multiplyScalar(scale);
59:   camera.far = 100 * scale;
60:   camera.updateProjectionMatrix();
61:   orbit.target.set(centerX, 1.5, centerZ);
62:   orbit.update();
63:   directionalLight.position.set(4, 8, 3).multiplyScalar(scale);
64:   Object.assign(directionalLight.shadow.camera, {
65:     left: -8 * scale, right: 8 * scale, top: 8 * scale,
66:     bottom: -8 * scale, far: 25 * scale,
67:   });
68:   directionalLight.shadow.camera.updateProjectionMatrix();
```

```
69:   return { renderer, scene, camera, orbit, directionalLight, bounds, centerX, centerZ };
70: }
```

src/room/loadRoom.js

> Room-source selection and validation: preset, custom, then default.

```
 1: import { buildCustomRoom } from "../custom_model.js";
 2: import { buildDefaultRoom } from "./defaultRoom.js";
 3: import { buildPresetRoom } from "./presetRoom.js";
 4:
 5: /** Loads sessionStorage room data or returns the default room when absent or invalid. */
 6: export function loadRoom() {
 7:   const preset = sessionStorage.getItem("presetRoom");
 8:   if (preset) {
 9:     try {
10:       return buildPresetRoom(JSON.parse(preset));
11:     } catch (error) {
12:       console.error("Bad presetRoom data:", error);
13:       sessionStorage.removeItem("presetRoom");
14:     }
15:   }
16:
17:   const saved = sessionStorage.getItem("customRoom");
18:   if (saved) {
19:     try {
20:       const { points, pillars } = JSON.parse(saved);
21:       if (points && points.length >= 3) {
22:         return buildCustomRoom(points, pillars ?? []);
23:       }
24:     } catch (error) {
25:       console.error("Bad customRoom data:", error);
26:     }
27:   }
28:   return buildDefaultRoom();
29: }
```

src/room/defaultRoom.js

> Fallback 10-by-10 room with four walls and a floor.

```
   1: import { BoxGeometry, Group, Mesh, MeshStandardMaterial, Vector3 } from "three";
   2:
   3: /** Builds and returns the fallback 10 by 10 room, with dimensions measured in scene units. */
   4: export function buildDefaultRoom() {
   5:   const W=10,D=10,H=4,T=.2,floorMat=new MeshStandardMaterial({color:0x8b5a2b}),backMat=new
MeshStandardMaterial({color:0xd9e4f5}),frontMat=new MeshStandardMaterial({color:0xf2d4d4}),leftMat=new
MeshStandardMaterial({color:0xf5e6d3}),rightMat=new MeshStandardMaterial({color:0xd4f2d9}),group=new Group(),walls=[];
   6:   function addPart(w,h,d,x,y,z,material){const mesh=new Mesh(new
BoxGeometry(w,h,d),material);mesh.position.set(x,y,z);mesh.receiveShadow=true;group.add(mesh);return mesh;}
   7:   function addWall(w,h,d,x,y,z,normal,material){const
mesh=addPart(w,h,d,x,y,z,material);walls.push({mesh,normal:new Vector3(...normal)});}
   8:   addPart(W,T,D,0,-T/2,0,floorMat);
   9:   addWall(W,H,T,0,H/2,-D/2,[0,0,-1],backMat);addWall(W,H,T,0,H/2,D/2,[0,0,1],frontMat);addWall(T,H,D-T,-W/2,H/2,0,
[-1,0,0],leftMat);addWall(T,H,D-T,W/2,H/2,0,[1,0,0],rightMat);
  10:   group.userData.walls=walls;group.userData.outline=[{x:-W/2,z:-D/2},{x:W/2,z:-D/2},{x:W/2,z:D/2},{x:-
W/2,z:D/2}];group.userData.pillars=[];group.userData.wallThickness=T;return group;
  11: }
```

src/room/presetRoom.js

> Preset rectangle, square, and cylinder conversion and dimension validation.

```
 1: import { buildCustomRoom } from "../custom_model.js";
 2:
 3: const PIXELS_PER_METRE = 50;
 4:
 5: function rectanglePoints(width, depth) {
 6:     const halfWidth = width * PIXELS_PER_METRE / 2;
 7:     const halfDepth = depth * PIXELS_PER_METRE / 2;
 8:     return [
 9:         { x: -halfWidth, y: -halfDepth },
10:         { x: halfWidth, y: -halfDepth },
11:         { x: halfWidth, y: halfDepth },
12:         { x: -halfWidth, y: halfDepth }
13:     ];
14: }
15:
16: function cylinderPoints(radius) {
17:     const points = [];
18:     const radiusInPixels = radius * PIXELS_PER_METRE;
19:
20:     for (let index = 0; index < 64; index += 1) {
21:         const angle = (index / 64) * Math.PI * 2;
22:         points.push({
23:             x: Math.cos(angle) * radiusInPixels,
24:             y: Math.sin(angle) * radiusInPixels
25:         });
26:     }
27:
28:     return points;
29: }
30:
31: /** Builds a room from the dimensions saved by the preset form. */
32: export function buildPresetRoom(preset) {
33:     const width = Number(preset.width);
34:     const depth = Number(preset.depth);
35:     const length = Number(preset.length);
36:     const radius = Number(preset.radius);
37:
38:     if (preset.shape === "cylinder" && radius > 0) {
39:         return buildCustomRoom(cylinderPoints(radius), []);
40:     }
41:
42:     if (preset.shape === "square" && length > 0) {
43:         return buildCustomRoom(rectanglePoints(length, length), []);
44:     }
45:
46:     if (width > 0 && depth > 0) {
47:         return buildCustomRoom(rectanglePoints(width, depth), []);
48:     }
49:
50:     throw new Error("Invalid preset room dimensions");
51: }
```

src/room/wallVisibility.js

> Camera-side wall visibility updates.

```
 1: import { Vector3 } from "three";
 2:
 3: const _wallPos = new Vector3();
 4: const _normal = new Vector3();
 5: const _toCam = new Vector3();
 6:
 7: /** Updates room wall visibility according to which side of each wall the camera occupies. */
 8: export function updateWallVisibility(room, camera) {
 9:   room.updateMatrixWorld();
10:   for (const { mesh, normal } of room.userData.walls ?? []) {
11:     mesh.getWorldPosition(_wallPos);
12:     _normal.copy(normal).transformDirection(mesh.parent.matrixWorld);
13:     _toCam.copy(camera.position).sub(_wallPos);
14:     mesh.visible = _toCam.dot(_normal) <= 0;
15:   }
16: }
```

src/furniture/catalog.js

> Built-in furniture metadata, now including the metal arm chair.

```
/** Plain catalog of furniture models and their display and sizing metadata. */
export const CATALOG = [
  { id: "sofa", label: "Sofa", path: "../models/black_sofa.glb", width: 3.5 },
  { id: "table", label: "Table", path: "../models/table.glb", width: 2.5 },
  { id: "bed", label: "Bed", path: "../models/bed.glb", width: 3 },
  { id: "lamp", label: "Lamp", path: "../models/lamp.glb", height: 2.5 },
  { id: "desktop", label: "Desktop", path: "../models/desktop.glb", width: 2 },
  { id: "chair", label: "metal arm chair", path: "../models/metal_arm_chair.glb", width: 1.5 },

];
```

src/furniture/loader.js

> Built-in model loading plus memory-file GLB parsing, caching, sizing, centering, and shadow setup.

```
import { Box3, Group, Vector3 } from "three";
import { GLTFLoader } from "three/addons/loaders/GLTFLoader.js";

const loader = new GLTFLoader();
const modelCache = new Map();
const importedModelCache = new WeakMap();

/** Loads a model path, sharing in-flight and completed loads by path. */
export function loadModel(path) {
  if (!modelCache.has(path)) {
    modelCache.set(
      path,
      loader.loadAsync(path).then((gltf) => gltf.scene).catch((error) => {
        modelCache.delete(path);
        throw error;
      }),
    );
  }
  return modelCache.get(path);
}

/** Builds a centered furniture pivot using catalog sizing metadata in scene units. */
export async function buildFurniture(entry) {
  const source = await loadModel(entry.path);
  return buildFurniturePivot(source, entry);
}

/** Parses a GLB File and builds its furniture pivot using entry dimensions in scene units. */
export async function buildFurnitureFromFile(file, entry = { width: 2 }) {
  if (!file.name.toLowerCase().endsWith(".glb")) {
    throw new Error("Choose a .glb file");
  }
  if (file.size === 0) {
    throw new Error("The selected file is empty");
  }

  if (!importedModelCache.has(file)) {
    importedModelCache.set(
      file,
      file.arrayBuffer()
        .then((data) => loader.parseAsync(data, ""))
        .then((gltf) => gltf.scene)
        .catch((error) => {
          importedModelCache.delete(file);
          throw error;
        }),
    );
  }
  const source = await importedModelCache.get(file);
  return buildFurniturePivot(source, entry);
}

function buildFurniturePivot(source, entry) {
  const model = source.clone(true);
  model.traverse((child) => {
    if (child.isMesh) {
      child.castShadow = true;
      child.receiveShadow = true;
    }
  });

  model.rotation.y = entry.rotationY ?? 0;
  let box = new Box3().setFromObject(model);
  const size = box.getSize(new Vector3());
  if (entry.file && (box.isEmpty() || !Number.isFinite(size.x) || size.x <= 0)) {
    throw new Error("The GLB file has no usable model geometry");
  }
  const scale = entry.height ? entry.height / size.y : entry.width / size.x;
```

```
  model.scale.setScalar(scale);
  box = new Box3().setFromObject(model);
  const center = box.getCenter(new Vector3());
  model.position.x -= center.x;
  model.position.z -= center.z;
  model.position.y -= box.min.y;

  const pivot = new Group();
  pivot.add(model);
  pivot.userData.isFurniture = true;
  pivot.userData.entry = entry;
  return pivot;
}
```

src/furniture/placement.js

> Nearest valid initial placement search at zero and ninety degrees.

```
 1: import { GRID_STEP } from "../config.js";
 2: import { footprint, fitsInRoom } from "../physics/collision.js";
 3: import { rectsOverlap } from "../physics/geometry.js";
 4:
 5: /** Finds the valid floor position nearest the room bounds center, trying 0 and 90 degree rotations. */
 6: export function findFreeSpot(pivot, room, furniture) {
 7:   const others = furniture.map(footprint);
 8:   let best = null;
 9:   for (const rotation of [0, Math.PI / 2]) {
10:     pivot.rotation.y = rotation;
11:     pivot.position.set(0, 0, 0);
12:     const relativeFootprint = footprint(pivot);
13:     for (let x = room.bounds.minX; x <= room.bounds.maxX; x += GRID_STEP) {
14:       for (let z = room.bounds.minZ; z <= room.bounds.maxZ; z += GRID_STEP) {
15:         const distance = Math.hypot(x - room.centerX, z - room.centerZ);
16:         if (best && distance >= best.distance) continue;
17:         const rect = {
18:           minX: relativeFootprint.minX + x,
19:           maxX: relativeFootprint.maxX + x,
20:           minZ: relativeFootprint.minZ + z,
21:           maxZ: relativeFootprint.maxZ + z,
22:         };
23:         if (!fitsInRoom(rect, room)) continue;
24:         if (others.some((other) => rectsOverlap(rect, other))) continue;
25:         best = { x, z, rot: rotation, distance };
26:       }
27:     }
28:   }
29:   return best;
30: }
```

src/furniture/store.js

> In-memory furniture collection.

```
 1: /** Creates the per-app furniture collection used by placement and interaction modules. */
 2: export function createFurnitureStore() {
 3:   const items = [];
 4:   return {
 5:     items,
 6:     add(item) {
 7:       items.push(item);
 8:     },
 9:     remove(item) {
10:       const index = items.indexOf(item);
11:       if (index !== -1) items.splice(index, 1);
12:     },
13:   };
14: }
```

src/physics/geometry.js

> Pure point, segment, polygon, and rectangle geometry predicates.

```
 1: /** Tests whether floor point (x, z) lies inside polygon vertices using x/z coordinates. */
 2: export function pointInPolygon(x, z, poly) {
 3:   let inside = false;
 4:   for (let i = 0, j = poly.length - 1; i < poly.length; j = i++) {
 5:     const a = poly[i];
 6:     const b = poly[j];
 7:     if (
 8:       (a.z > z) !== (b.z > z) &&
 9:       x < ((b.x - a.x) * (z - a.z)) / (b.z - a.z) + a.x
10:     ) {
11:       inside = !inside;
12:     }
13:   }
14:   return inside;
15: }
16:
17: /** Tests whether segment endpoints a/b cross the interior of axis-aligned floor rectangle r. */
18: export function segmentHitsRect(a, b, r) {
19:   const dx = b.x - a.x;
20:   const dz = b.z - a.z;
21:   const p = [-dx, dx, -dz, dz];
22:   const q = [a.x - r.minX, r.maxX - a.x, a.z - r.minZ, r.maxZ - a.z];
23:   let t0 = 0;
24:   let t1 = 1;
25:
26:   for (let i = 0; i < 4; i++) {
27:     if (p[i] === 0) {
28:       if (q[i] <= 0) return false;
29:     } else {
30:       const t = q[i] / p[i];
31:       if (p[i] < 0) {
32:         if (t > t1) return false;
33:         t0 = Math.max(t0, t);
34:       } else {
35:         if (t < t0) return false;
36:         t1 = Math.min(t1, t);
37:       }
38:     }
39:   }
40:   return t1 - t0 > 1e-9;
41: }
42:
43: /** Tests whether a floor polygon overlaps the interior of rectangle r. */
44: export function polygonHitsRect(poly, r) {
45:   for (let i = 0; i < poly.length; i++) {
46:     if (segmentHitsRect(poly[i], poly[(i + 1) % poly.length], r)) return true;
47:   }
48:   return pointInPolygon((r.minX + r.maxX) / 2, (r.minZ + r.maxZ) / 2, poly);
49: }
50:
51: /** Returns whether axis-aligned floor rectangles a and b overlap beyond EPS. */
52: export function rectsOverlap(a, b, EPS = 1e-4) {
53:   return (
54:     a.minX < b.maxX - EPS &&
55:     a.maxX > b.minX + EPS &&
56:     a.minZ < b.maxZ - EPS &&
57:     a.maxZ > b.minZ + EPS
58:   );
59: }
```

src/physics/collision.js

> Furniture footprints, room fitting, collision checks, nudging, offsets, and sliding movement.

```
 1: import { Box3 } from "three";
 2: import { EPS } from "../config.js";
 3: import {
 4:   pointInPolygon,
 5:   polygonHitsRect,
 6:   rectsOverlap,
 7:   segmentHitsRect,
 8: } from "./geometry.js";
 9:
10: /** Builds an object's world-space floor footprint, caching bounds for its current rotation. */
11: export function footprint(obj) {
12:   let cached = obj.userData.fp;
13:   if (!cached || cached.rot !== obj.rotation.y) {
14:     obj.updateMatrixWorld(true);
15:     const box = new Box3().setFromObject(obj);
16:     cached = obj.userData.fp = {
17:       rot: obj.rotation.y,
18:       minX: box.min.x - obj.position.x,
19:       maxX: box.max.x - obj.position.x,
20:       minZ: box.min.z - obj.position.z,
21:       maxZ: box.max.z - obj.position.z,
22:     };
23:   }
24:   return {
25:     minX: cached.minX + obj.position.x,
26:     maxX: cached.maxX + obj.position.x,
27:     minZ: cached.minZ + obj.position.z,
28:     maxZ: cached.maxZ + obj.position.z,
29:   };
30: }
31:
32: /** Tests whether floor rectangle r fits within room outline and outside pillars. */
33: export function fitsInRoom(r, room) {
34:   const margin = room.wallThickness / 2;
35:   const grown = {
36:     minX: r.minX - margin,
37:     maxX: r.maxX + margin,
38:     minZ: r.minZ - margin,
39:     maxZ: r.maxZ + margin,
40:   };
41:
42:   for (let i = 0; i < room.outline.length; i++) {
43:     if (segmentHitsRect(room.outline[i], room.outline[(i + 1) % room.outline.length], grown)) {
44:       return false;
45:     }
46:   }
47:   if (!pointInPolygon((r.minX + r.maxX) / 2, (r.minZ + r.maxZ) / 2, room.outline)) {
48:     return false;
49:   }
50:   return !room.pillars.some((pillar) => polygonHitsRect(pillar, r));
51: }
52:
53: /** Tests whether item has a free footprint in room and among the furniture list. */
54: export function isFree(item, room, furniture) {
55:   const rect = footprint(item);
56:   if (!fitsInRoom(rect, room)) return false;
57:   return !furniture.some(
58:     (other) => other !== item && rectsOverlap(rect, footprint(other), EPS),
59:   );
60: }
61:
62: /** Moves item to the nearest free spot within radius floor units after rotation. */
63: export function nudgeToFree(item, radius, room, furniture) {
64:   const x0 = item.position.x;
65:   const z0 = item.position.z;
66:   let best = null;
67:
68:   for (let dx = -radius; dx <= radius + EPS; dx += 0.1) {
```

```
    for (let dz = -radius; dz <= radius + EPS; dz += 0.1) {
      const distance = Math.hypot(dx, dz);
      if (best && distance >= best.distance) continue;
      item.position.set(x0 + dx, 0, z0 + dz);
      if (isFree(item, room, furniture)) {
        best = { x: x0 + dx, z: z0 + dz, distance };
      }
    }
  }

  if (!best) {
    item.position.set(x0, 0, z0);
    return false;
  }
  item.position.set(best.x, 0, best.z);
  return true;
}

/** Attempts floor offset (ox, oz), restoring the original position if blocked. */
export function tryOffset(item, ox, oz, room, furniture) {
  const x0 = item.position.x;
  const z0 = item.position.z;
  let fraction = 1;
  for (let i = 0; i < 6; i++) {
    item.position.set(x0 + ox * fraction, 0, z0 + oz * fraction);
    if (isFree(item, room, furniture)) return true;
    fraction /= 2;
  }
  item.position.set(x0, 0, z0);
  return false;
}

/** Moves item toward target floor coordinates while sliding along blocked obstacles. */
export function moveWithCollision(item, targetX, targetZ, room, furniture) {
  const dx = targetX - item.position.x;
  const dz = targetZ - item.position.z;
  const steps = Math.max(1, Math.ceil(Math.hypot(dx, dz) / 0.2));
  const stepX = dx / steps;
  const stepZ = dz / steps;

  for (let i = 0; i < steps; i++) {
    if (!tryOffset(item, stepX, stepZ, room, furniture) &&
        !tryOffset(item, stepX, 0, room, furniture)) {
      tryOffset(item, 0, stepZ, room, furniture);
    }
  }
}
```

src/interaction/drag.js

> Floor-plane pointer dragging and R-key rotation with collision recovery.

```
import { Plane, Raycaster, Vector2, Vector3 } from "three";
import { moveWithCollision, nudgeToFree } from "../physics/collision.js";
import { showMessage } from "../ui/toast.js";

/** Installs floor dragging, click/key drop, and R rotation controls; distances use scene units. */
export function installDrag({ renderer, camera, orbit, furniture, room }) {
  const raycaster = new Raycaster();
  const pointer = new Vector2();
  const floorPlane = new Plane(new Vector3(0, 1, 0), 0);
  const hitPoint = new Vector3();
  const dragOffset = new Vector3();
  let dragging = false;
  let activeItem = null;

  function setPointer(event) {
    const rect = renderer.domElement.getBoundingClientRect();
    pointer.x = ((event.clientX - rect.left) / rect.width) * 2 - 1;
    pointer.y = -((event.clientY - rect.top) / rect.height) * 2 + 1;
    raycaster.setFromCamera(pointer, camera);
  }
  function pick() {
    const hits = raycaster.intersectObjects(furniture, true);
    if (hits.length === 0) return null;
    let object = hits[0].object;
    while (object && !object.userData.isFurniture) object = object.parent;
    return object;
  }
  function startDrag(item, event) {
    setPointer(event);
    if (!raycaster.ray.intersectPlane(floorPlane, hitPoint)) return;
    activeItem = item;
    dragging = true;
    orbit.enabled = false;
    renderer.domElement.style.cursor = "grabbing";
    dragOffset.copy(hitPoint).sub(item.position);
    showMessage("Click or press a key to drop (R rotates)");
  }
  function stopDrag() {
    if (!dragging) return;
    dragging = false;
    activeItem = null;
    orbit.enabled = true;
    renderer.domElement.style.cursor = "auto";
  }

  renderer.domElement.addEventListener("pointermove", (event) => {
    if (!dragging) return;
    setPointer(event);
    if (raycaster.ray.intersectPlane(floorPlane, hitPoint)) {
      moveWithCollision(
        activeItem,
        hitPoint.x - dragOffset.x,
        hitPoint.z - dragOffset.z,
        room,
        furniture,
      );
    }
  });
  window.addEventListener("pointerdown", stopDrag);
  window.addEventListener("keydown", (event) => {
    if (!dragging) return;
    if (event.key.toLowerCase() === "r" && !event.ctrlKey && !event.metaKey) {
      if (event.repeat) return;
      const oldRot = activeItem.rotation.y;
      const oldX = activeItem.position.x;
      const oldZ = activeItem.position.z;
      activeItem.rotation.y += Math.PI / 2;
      if (!nudgeToFree(activeItem, 1.5, room, furniture)) {
```

```
69:         activeItem.rotation.y = oldRot;
70:         activeItem.position.set(oldX, 0, oldZ);
71:         showMessage("No room to rotate here");
72:       }
73:       return;
74:     }
75:     stopDrag();
76:   });
77:
78:   return { startDrag, stopDrag, isDragging: () => dragging, pick, setPointer };
79: }
```

src/interaction/contextMenu.js

> Right-click Drag and Remove menu.

```
/** Installs the furniture Drag/Remove context menu and returns its DOM element. */
export function installContextMenu({ renderer, furniture, scene, drag, store }) {
  let menu = null;
  let target = null;
  function close() {
    if (menu) menu.hidden = true;
    target = null;
  }
  function open(x, y, item) {
    target = item;
    menu.hidden = false;
    menu.style.left = Math.min(x, window.innerWidth - menu.offsetWidth - 4) + "px";
    menu.style.top = Math.min(y, window.innerHeight - menu.offsetHeight - 4) + "px";
  }

  const style = document.createElement("style");
  style.textContent = `
    .ctx-menu { position: fixed; z-index: 20; width: 120px; background: #fff; border-radius: 6px; box-shadow: 0 2px 8px rgba(0,0,0,.35); font-family: system-ui, sans-serif; overflow: hidden; }
    .ctx-menu[hidden] { display: none; }
    .ctx-menu button { display: block; width: 100%; padding: 8px 14px; border: 0; background: none; color: #222; text-align: left; font-size: 14px; cursor: pointer; }
    .ctx-menu button:hover { background: #eef3fb; }
    .ctx-menu button.danger { color: #c62828; }
  `;
  document.head.append(style);

  menu = document.createElement("div");
  menu.className = "ctx-menu";
  menu.hidden = true;
  const dragButton = document.createElement("button");
  dragButton.textContent = "Drag";
  dragButton.addEventListener("click", (event) => {
    const item = target;
    close();
    if (item) drag.startDrag(item, event);
  });
  const removeButton = document.createElement("button");
  removeButton.textContent = "Remove";
  removeButton.className = "danger";
  removeButton.addEventListener("click", () => {
    const item = target;
    close();
    if (item) {
      scene.remove(item);
      store.remove(item);
    }
  });
  menu.append(dragButton, removeButton);
  document.body.append(menu);

  renderer.domElement.addEventListener("contextmenu", (event) => {
    event.preventDefault();
    if (drag.isDragging()) return;
    drag.setPointer(event);
    const item = drag.pick();
    if (item) open(event.clientX, event.clientY, item);
    else close();
  });
  window.addEventListener("pointerdown", (event) => {
    if (!menu.contains(event.target)) close();
  });
  window.addEventListener("keydown", (event) => {
    if (event.key === "Escape") close();
  });
  return menu;
}
```

src/ui/addMenu.js

> Built-in Add menu plus Import button and hidden .glb file input.

```
/** Builds the Add/Import menu, calling onAdd(entry) or onImport(file, addEntry) as selected. */
export function buildAddMenu(catalog, onAdd, onImport) {
  const style = document.createElement("style");
  style.textContent = `
    .add-menu { position: fixed; top: 12px; left: 12px; z-index: 10; display: flex; gap: 6px; font-family: system-ui, sans-serif; }
    .add-menu > button { padding: 8px 14px; border: 0; border-radius: 6px; background: #fff; color: #222; font-size: 14px; cursor: pointer; box-shadow: 0 1px 4px rgba(0,0,0,.3); }
    .add-list { position: absolute; top: 28px; left: 0; margin-top: 6px; width: 160px; max-height: 160px; overflow-y: auto; background: #fff; border-radius: 6px; box-shadow: 0 2px 8px rgba(0,0,0,.3); }
    .add-list[hidden] { display: none; }
    .add-list button { display: block; width: 100%; padding: 8px 14px; border: 0; background: none; color: #222; text-align: left; font-size: 14px; cursor: pointer; }
    .add-list button:hover { background: #eef3fb; }
    #toast { position: fixed; top: 12px; left: 50%; transform: translateX(-50%); padding: 8px 14px; border-radius: 6px; background: rgba(0,0,0,.8); color: #fff; font: 14px system-ui, sans-serif; z-index: 10; opacity: 0; transition: opacity .2s; pointer-events: none; }
  `;
  document.head.append(style);

  const wrap = document.createElement("div");
  wrap.className = "add-menu";
  const button = document.createElement("button");
  button.textContent = "Add ▾";
  const importButton = document.createElement("button");
  importButton.textContent = "Import";
  const fileInput = document.createElement("input");
  fileInput.type = "file";
  fileInput.accept = ".glb,model/gltf-binary";
  fileInput.hidden = true;
  const list = document.createElement("div");
  list.className = "add-list";
  list.hidden = true;

  function addEntry(entry) {
    const option = document.createElement("button");
    option.textContent = entry.label;
    option.addEventListener("click", () => {
      list.hidden = true;
      onAdd(entry);
    });
    list.append(option);
  }

  for (const entry of catalog) addEntry(entry);
  button.addEventListener("click", () => (list.hidden = !list.hidden));
  importButton.addEventListener("click", () => fileInput.click());
  fileInput.addEventListener("change", () => {
    const file = fileInput.files?.[0];
    fileInput.value = "";
    if (file) onImport(file, addEntry);
  });
  document.addEventListener("pointerdown", (event) => {
    if (!wrap.contains(event.target)) list.hidden = true;
  });

  wrap.append(button, importButton, list, fileInput);
  document.body.append(wrap);
  return { addEntry };
}
```

src/ui/toast.js

> Transient user feedback messages.

```
let toastTimer = null;

/** Displays a user-facing message for 2.5 seconds. */
export function showMessage(text) {
  let toast = document.getElementById("toast");
  if (!toast) {
    toast = document.createElement("div");
    toast.id = "toast";
    document.body.append(toast);
  }
  toast.textContent = text;
  toast.style.opacity = "1";
  clearTimeout(toastTimer);
  toastTimer = setTimeout(() => (toast.style.opacity = "0"), 2500);
}
```

package.json

> Three.js runtime dependency.

```
1: {
2:   "dependencies": {
3:     "three": "^0.187.0"
4:   }
5: }
```

readme.md

> Architecture and catalog-extension notes.

```
   1: # 3D Room Customizer
   2:
   3: ## ES module layout
   4:
   5: - `src/main.js` wires the app modules together.
   6: - `src/config.js` contains placement and display settings.
   7: - `src/core/` creates the renderer, camera, controls, lights, and helpers.
   8: - `src/room/` loads drawn or default rooms and updates wall visibility. `src/custom_model.js` builds rooms drawn on the canvas page.
   9: - `src/physics/geometry.js` contains pure floor polygon and rectangle helpers; `collision.js` applies them to room and furniture data.
  10: - `src/furniture/` contains the catalog, GLTF model loader, furniture store, and placement search.
  11: - `src/interaction/` handles dragging, rotation, and the right-click menu.
  12: - `src/ui/` contains the Add menu and toast message.
  13: - `src/canvas.js` remains a separate page script and saves the drawn room in session storage.
  14:
  15: Dependencies flow from core room and physics utilities toward furniture; interaction and UI use those modules, and `main.js` connects the app. `physics/geometry.js` has no Three.js or DOM dependency.
  16:
  17: ## Adding furniture
  18:
  19: Add one object to the plain `CATALOG` array in `src/furniture/catalog.js`. Provide a unique `id`, the displayed `label`, a model `path`, and either a target `width` or `height` in scene units. The Add menu and loader consume the entry automatically. Keep model paths valid relative to the page URL, as with the existing entries.
  20:
  21: ## Items noticed and left unchanged
  22:
  23: - The fallback path assumes room builders provide a non-empty `userData.outline`; malformed room geometry can still fail while bounds are calculated.
  24: - `canvas.js` retains its original drawing and session storage flow, including its console logging.
  25: - The GLTF catalog paths use `../models/...` as in the existing app; their resolution depends on the static server's page URL layout.
```